ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Система управления складом и кинопроизводством

---

1. ЧТО ЭТО ЗА ПРОЕКТ

Веб-приложение для съёмочных групп. Одна часть — склад реквизита и костюмов: принять, выдать, вернуть, следить за состоянием. Другая часть — площадка: документы съёмочного дня, списки, заявки. Всё в одном месте, у каждого своя роль и права.

Технически: Fullstack SPA с разделением на два мира: `warehouse` — управление складскими активами, `production` — работа съёмочной площадки. Единая авторизация, ролевая модель на 30+ ролей, REST API.

---

2. ИЗ ЧЕГО СОСТОИТ

Три части — интерфейс который видит пользователь, сервер который всё обрабатывает, и база данных где всё хранится. Плюс отдельное хранилище для файлов.

Технически: Монорепо с двумя пакетами — `frontend` и `backend`. Деплоятся раздельно: бэкенд на Railway, фронтенд на Vercel. В продакшне бэкенд также отдаёт собранный фронтенд как статику.

Компонент	Технология	Зачем
Frontend	React 19 + Vite	Интерфейс пользователя
Backend	Node.js + Express 5	API, бизнес-логика
База данных	PostgreSQL	Хранение всех данных
Файлы	AWS S3	Документы, фото активов
Деплой бэкенда	Railway	Хостинг сервера + БД
Деплой фронтенда	Vercel	Хостинг интерфейса
Пуш-уведомления	web-push (VAPID)	Браузерные уведомления

3. ТЕХНОЛОГИИ

Frontend

Интерфейс написан на React — популярная библиотека для создания сайтов. Vite — это инструмент который собирает код в быстрый сайт. Никаких готовых визуальных библиотек — всё нарисовано с нуля.

Технически: React 19 с хуками, React Router 7 для навигации, Lucide React для иконок. Сборка через Vite 8. ESLint для проверки кода. Без UI-фреймворков (no MUI, no Tailwind).

Библиотека	Версия	Роль
react	19.2	UI-фреймворк
react-router-dom	7.13	Маршрутизация
lucide-react	1.0	Иконки
vite	8.0	Сборщик

Backend

Сервер написан на JavaScript (Node.js). Express — это каркас который принимает запросы от браузера и отвечает на них. PostgreSQL — база данных, как большая таблица Excel но намного мощнее.

Технически: Node.js CommonJS модули, Express 5 с async error handling. PostgreSQL через драйвер `pg`. Авторизация через JWT (jsonwebtoken + bcrypt для паролей). Автоматические SQL-миграции при старте.

Библиотека	Роль
express 5	HTTP-сервер, роутинг
pg	Подключение к PostgreSQL
jsonwebtoken	JWT токены авторизации
bcrypt	Хеширование паролей
multer	Загрузка файлов
@aws-sdk/client-s3	Работа с S3 хранилищем
exceljs	Генерация Excel-отчётов
pdf-lib + pdf-parse	Создание и чтение PDF
mammoth	Чтение Word документов
web-push	Браузерные push-уведомления

4. РОЛИ ПОЛЬЗОВАТЕЛЕЙ

В системе два типа людей — складские (работают со складом) и съёмочные (работают на площадке). У каждого свои права: кто-то может только смотреть, кто-то — выдавать вещи, кто-то — удалять и управлять всем.

Технически: Роли хранятся в `frontend/src/constants/roles.js` как объект `ROLES`. Каждая роль имеет поле `world` (`warehouse` или `production`) и набор булевых флагов прав. Проверка ролей на уровне React Router (компоненты `WarehouseRoute`, `ProductionRoute`) и на бэкенде через middleware `checkRole`.

Складской мир (world: warehouse)

Роль	Ключ	Основные права
Директор склада	`warehouse_director`	Всё: выдача, списание, аналитика, удаление, приглашения

Роль	Ключ	Основные права
Зам директора склада	`warehouse_deputy`	Выдача, просмотр истории, публичные ссылки
Сотрудник склада	`warehouse_staff`	Только выдача и публичные ссылки

Мир площадки (world: production)

Роль	Ключ	Основные права
Продюсер	`producer`	Аналитика, просмотр всего, приглашение всех
Директор проекта	`project_director`	Загрузка КПП, сценария, вызывного; приглашения
Художник-постановщик	`production_designer`	Загрузка доков, все списки
Реквизитор	`props_master`	Загрузка доков, списки: реквизит, костюмы, бутафория, авто
Костюмер	`costumer`	Загрузка доков, списки костюмов
Гримёр	`makeup_artist`	Загрузка доков, список грима, отчёт о закупках
Декоратор	`decorator`	Загрузка доков, списки декора
Постановщик трюков	`stunt_coordinator`	Загрузка доков, список трюков
Пиротехник	`pyrotechnician`	Загрузка доков, список пиротехники
Режиссёр-постановщик	`director`	Чтение всех доков и списков
Оператор, Гафер, Кастинг	разные	Чтение КПП, сценария, вызывного
Водитель, Плейбэкер, Механик	разные	Чтение КПП и вызывного, без уведомлений о версиях

5. КАК ДАННЫЕ ДВИЖУТСЯ

Пользователь нажимает кнопку в браузере → браузер отправляет запрос на сервер → сервер проверяет кто это и что ему можно → достаёт данные из базы → отправляет обратно → браузер показывает результат.

Технически: Клиент отправляет HTTP-запрос с JWT в заголовке `Authorization: Bearer <token>`. Middleware `verifyJWT` проверяет токен и кладёт `req.user`. Затем опционально `checkRole()` проверяет права. Контроллер делает запрос в PostgreSQL через `pool.query()`. Ответ — JSON.

```
Браузер → HTTPS запрос + JWT токен
   → Express middleware: verifyJWT
   → Express middleware: checkRole (если нужно)
   → Route handler
   → PostgreSQL query
   → JSON ответ
   → React обновляет UI
```

---

6. КЛЮЧЕВЫЕ МОДУЛИ

Frontend — структура папок

Код интерфейса разбит по смыслу — отдельно компоненты склада, отдельно площадки, отдельно общие. Константы (роли, статусы, категории) вынесены отдельно.

Технически: `src/components/` разбит на `warehouse/`, `production/`, `analytics/`, `auth/`, `movement/`, `shared/`, `dev/`. Общая логика в `hooks/`, утилиты в `utils/`, константы в `constants/`.

Папка	Что внутри
`components/warehouse/`	Склад: дашборд, единицы, ячейки, заявки, акты, команда
`components/production/`	Площадка: документы, списки, заявки, сотрудники, просмотр склада
`components/analytics/`	Аналитика директора склада и продюсера
`components/movement/`	Выдача и возврат активов
`components/auth/`	Логин, регистрация, приглашение, восстановление пароля
`components/shared/`	Уведомления, профиль, Toast-сообщения
`constants/`	roles.js, statuses.js, categories.js
`hooks/`	useAuth — авторизация через Context API
`services/`	API-запросы к бэкенду
`utils/`	getHomeRoute и другие вспомогательные функции

Backend — API маршруты

Сервер умеет отвечать на разные запросы — про авторизацию, про вещи на складе, про документы, про аренду и так далее. Каждый тип запросов живёт в своём файле.

Технически: `src/routes/` содержит отдельный роутер на каждую сущность, подключённый в `index.js` через `app.use()`.

Маршрут	Файл	Отвечает за
`/auth`	routes/auth	Логин, регистрация, токены
`/invites`	routes/invites	Приглашения по ссылке
`/units`	routes/units	Активы склада (CRUD)
`/warehouses`	routes/warehouses	Склады и ячейки
`/requests`	routes/requests	Заявки на выдачу
`/issuances`	routes/issuances	Выдачи и возвраты
`/documents`	routes/documents	КПП, сценарии, вызывные
`/rent`	routes/rent	Аренда (с JWT)
`/public`	routes/publicRent	Публичные страницы (без JWT)
`/analytics`	routes/analytics	Аналитика и отчёты
`/team`	routes/team	Управление командой
`/lists`	routes/lists	Списки площадки
`/debts`	routes/debts	Просроченные выдачи
`/projects`	index.js	Список и создание проектов
`/notifications`	index.js	Уведомления пользователя
`/push/*`	index.js	Push-подписки

Статусы активов

У каждой вещи на складе есть статус — где она сейчас находится и что с ней происходит.

Технически: Константы в `frontend/src/constants/statuses.js`. Статус хранится в БД как строка, отображается через `STATUS_LABEL`, красится через `STATUS_COLOR`.

Ключ	Название	Цвет
`on_stock`	На складе	Зелёный
`issued`	Выдано	Синий
`overdue`	Просрочено	Красный
`pending`	На утверждении	Жёлтый
`written_off`	Списано	Серый

Категории активов

16 категорий: Костюмы, Реквизит, Художественное наполнение, Бутафория, Автомобили, Мебель, Декор, Декорации, Техника, Осветительное оборудование, Звуковое оборудование, Камерное оборудование, Грим и косметика, Одежда, Украшения, Прочее.

---

7. КАК ЗАПУСТИТЬ ЛОКАЛЬНО

Нужно установить Node.js, создать базу данных PostgreSQL, прописать настройки в файле и запустить два процесса — сервер и интерфейс.

Технически:

Шаг 1 — Требования:

- Node.js 22.12.0 (версия зафиксирована в `.node-version`)
- PostgreSQL (любая актуальная версия)

Шаг 2 — Переменные окружения. Создай `backend/.env` по образцу `backend/.env.example`:

```
DATABASE_URL=postgresql://user:password@localhost:5432/dbname
JWT_SECRET=любая_длинная_строка
FRONTEND_URL=http://localhost:5173
AWS_BUCKET_NAME=...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=...
VAPID_PUBLIC_KEY=...
VAPID_PRIVATE_KEY=...
```

Шаг 3 — Установка зависимостей:

bash
```
cd backend && npm install
```

cd ../frontend && npm install

Шаг 4 — Запуск бэкенда (автоматически применит миграции):

bash
cd backend && npm run dev
Сервер стартует на http://localhost:3000

Шаг 5 — Запуск фронтенда:

bash
cd frontend && npm run dev
Интерфейс на http://localhost:5173
```

**Шаг 6 — Тестовые данные** (если есть):
```
http://localhost:5173/dev/seed
```

---

# 8. КАК УСТРОЕН ДЕПЛОЙ

**Простыми словами:** Бэкенд живёт на Railway — он сам следит за сервером и базой данных. Фронтенд живёт на Vercel — это специальный хостинг для интерфейсов. При каждом пуше в GitHub всё обновляется автоматически.

**Технически:**

| Сервис | Платформа | Конфиг |
|---|---|---|
| Backend + PostgreSQL | Railway | `railway.json`, `Dockerfile` |
| Frontend | Vercel | `vercel.json` |

Деплой бэкенда использует `Dockerfile` (переход с Nixpacks из-за ошибки с ENV). При старте контейнера автоматически применяются SQL-миграции из `backend/src/db/migrations/` — система сама отслеживает какие уже применены через таблицу `_migrations`.

В продакшне бэкенд отдаёт собранный фронтенд (`frontend/dist`) как статические файлы — одно приложение на одном домене.

---

# 9. КАРТА ПАПОК
```
/
├── backend/
│   ├── src/
│   │   ├── index.js   ← точка входа, все маршруты, миграции
│   │   ├── db/
│   │   │   ├── index.js   ← pool подключения к PostgreSQL
│   │   │   ├── migrate.js   ← запуск миграций
│   │   │   └── migrations/ ← SQL файлы миграций (применяются по порядку)
│   │   ├── routes/   ← отдельный файл на каждую сущность API
│   │   ├── middleware/
│   │   │   └── auth.js   ← verifyJWT, checkRole
│   │   └── services/
```
```

```
│   │   │   ├── push.js       ← отправка браузерных уведомлений
│   │   │   └── notifications.js ← проверка просрочек (каждые 30 мин)
│   ├── .env.example          ← шаблон переменных окружения
│   └── package.json
│
├── frontend/
│   ├── src/
│   │   ├── App.jsx           ← роутинг, защита маршрутов по ролям
│   │   ├── main.jsx          ← точка входа, AuthProvider, ToastProvider
│   │   ├── components/
│   │   │   ├── warehouse/    ← всё для складских ролей
│   │   │   ├── production/   ← всё для съёмочных ролей
│   │   │   ├── analytics/    ← дашборды аналитики
│   │   │   ├── movement/     ← выдача и возврат
│   │   │   ├── auth/         ← авторизация
│   │   │   ├── rent/         ← аренда и подписание
│   │   │   ├── shared/       ← общие компоненты
│   │   │   └── dev/          ← инструменты разработки (SeedPage)
│   │   ├── constants/
│   │   │   ├── roles.js      ← все роли и их права
│   │   │   ├── statuses.js   ← статусы активов
│   │   │   └── categories.js ← категории имущества
│   │   ├── hooks/
│   │   │   └── useAuth.js    ← глобальный стейт авторизации
│   │   ├── services/         ← fetch-запросы к API
│   │   └── utils/
│   │       └── getHomeRoute.js ← куда редиректить по роли
│   ├── index.html
│   └── package.json
│
├── Dockerfile                ← сборка для Railway
└── railway.json              ← конфиг деплоя
```

---

10. ЧТО СЛОМАЕТСЯ ЕСЛИ УБРАТЬ КАЖДЫЙ МОДУЛЬ

Простыми словами: Каждая часть системы важна по-своему. Вот что произойдёт если что-то отключить.

Модуль	Что сломается
`constants/roles.js`	Вся система прав — никто не сможет войти и попасть на нужную страницу
`hooks/useAuth.js`	Авторизация во всём приложении — все страницы станут недоступны
`middleware/auth.js`	Все защищённые API-эндпоинты перестанут работать
`db/migrations/`	База данных не создастся при новом деплое — приложение не запустится
`services/push.js`	Браузерные уведомления перестанут приходить
`services/notifications.js`	Просрочки не будут автоматически обнаруживаться (каждые 30 мин)
`routes/public`	Публичные ссылки на склад и страница подписания аренды упадут
AWS S3	Загрузка и просмотр файлов (документы, фото активов)

Модуль	Что сломается
	перестанут работать
`frontend/dist`	В продакшне (Railway) интерфейс не откроется
PostgreSQL	Полный отказ — всё приложение встанет
JWT_SECRET в .env	При рестарте сервера все активные сессии протухнут — все выйдут из системы
`components/dev/SeedPage`	Только потеря инструмента для тестовых данных — продакшн не затронет

11. СПЕЦИАЛЬНЫЕ МЕХАНИЗМЫ

Impersonate (просмотр от лица пользователя)

Простыми словами: Продюсер может «войти» в аккаунт другого пользователя чтобы посмотреть что тот видит. Кнопка «Вернуться» возвращает обратно.

Технически: При impersonate токен продюсера сохраняется в `sessionStorage` под ключами `producer_token` и `producer_user`. Компонент `ImpersonateBanner` в `App.jsx` показывает кнопку возврата. При клике восстанавливается оригинальный токен.

Публичные ссылки

Простыми словами: Склад можно открыть по специальной ссылке без входа в систему — например показать клиенту что есть на складе.

Технически: Маршрут `/public/warehouse/:token` не требует JWT. Обрабатывается отдельным роутером `routes/publicRent` без middleware авторизации.

Автоматические миграции

Простыми словами: При каждом запуске сервер сам проверяет нужно ли обновить структуру базы данных и делает это автоматически.

Технически: При старте `runMigrations()` читает все `.sql` файлы из `db/migrations/` отсортированные по имени, проверяет таблицу `_migrations` — не применённые выполняет в транзакции и фиксирует. Откат при ошибке.

Проверка просрочек

Простыми словами: Каждые 30 минут система автоматически проверяет есть ли вещи которые не вернули вовремя и отправляет уведомления.

Технически: `setInterval(checkOverdue, 30 * 60 * 1000)` в `index.js` вызывает сервис `services/notifications.js`. При обнаружении просрочек создаёт записи в таблице `notifications` и отправляет web-push подписчикам.